

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN

Universidad Tecnológica Nacional — Modalidad a Distancia

Materia: Programación 1

Trabajo Práctico Integrador (TPI)

Gestión de Datos de Países en Python: filtros, ordenamientos y estadísticas

Integrantes

Franco Moiana

Leonardo Macre

Fecha de entrega: 15 de junio de 2026

Índice

1. Marco Teórico	3
1.1. Listas	3
1.2. Diccionarios	3
1.3. Funciones	4
1.4. Condicionales	4
1.5. Ordenamientos	4
1.6. Estadísticas básicas	5
1.7. Archivos CSV	5
2. Decisiones Técnicas y Arquitectura	6
2.1. Estructura del proyecto	6
2.2. Diagramas de flujo	6
2.3. Ejemplos de ejecución	9
3. Dificultades y Conclusiones	10
4. Bibliografía / Webgrafía	11
5. Links	11

1. Marco Teórico

Para desarrollar este proyecto usamos varios conceptos que fuimos viendo a lo largo de la materia. Aquí explicamos cada uno y cómo lo aplicamos en el sistema.

1.1. Listas

Una lista es una de las estructuras de datos más básicas de Python. Sirve para guardar varios elementos juntos en una sola variable, en orden, y se puede acceder a cada uno por su posición. En nuestro programa, la variable **países** es una lista que va acumulando todos los países que se cargan desde el CSV. Elegimos listas porque nos permiten recorrer todos los países con un **for**, agregar nuevos con **append()** y pasarlos de función en función sin complicaciones.

```
países = []
países.append(nuevo_pais) # se agrega al final de la lista
```

1.2. Diccionarios

Un diccionario guarda información en pares clave-valor, algo parecido a una ficha donde cada campo tiene un nombre. En nuestro caso, cada país es un diccionario con cuatro campos: **nombre**, **poblacion**, **superficie** y **continente**. Usamos diccionarios porque el código queda mucho más claro que si usáramos listas con índices numéricos: escribir **pais['nombre']** es más fácil de entender que **pais[0]**.

```
pais = {
    'nombre': 'Argentina',
    'poblacion': 45376763,
    'superficie': 2780400,
    'continente': 'América'
}
```

1.3. Funciones

Las funciones son bloques de código que tienen un nombre y hacen una tarea específica. En este proyecto intentamos que cada función haga una sola cosa, como dice la consigna. Por ejemplo, **agregar_pais()** solo se encarga de agregar, **filtrar_por_continente()** solo filtra, y **guardar_csv()** solo escribe el archivo. Esto nos ayudó mucho cuando había que corregir algo, porque sabíamos exactamente dónde buscar.

1.4. Condicionales

Los condicionales (**if / elif / else**) le permiten al programa tomar decisiones según lo que pasa en cada momento. Los usamos en varios lados: para ver si el país que se quiere agregar ya existe, para validar que el usuario ingrese una opción válida del menú, y para decidir qué función ejecutar dependiendo de la opción elegida.

```
if pais['nombre'].lower() == nombre.lower():
    print('El país ya existe.')
    return False
```

1.5. Ordenamientos

Para ordenar la lista de países usamos la función **sorted()** de Python, que devuelve una nueva lista ordenada sin tocar la original. Con el parámetro **key** le decimos por qué campo ordenar, y con **reverse** elegimos si el orden es de menor a mayor o al revés. Para definir el campo usamos funciones lambda, que son una forma corta de escribir funciones simples.

```
sorted(paises, key=lambda pais: pais['poblacion'], reverse=False)
```

1.6. Estadísticas básicas

Las estadísticas nos permiten sacar información resumida del conjunto de datos. Implementamos varias: el país con más y menos población usando **max()** y **min()**, el promedio de población y superficie con **sum()** dividido por **len()**, y la cantidad de países por continente usando un diccionario como contador donde cada continente es una clave y el valor es cuántos países tiene.

```
promedio = sum(p['poblacion'] for p in paises) / len(paises)
```

1.7. Archivos CSV

CSV significa Comma-Separated Values, o sea valores separados por comas. Es un formato de archivo de texto muy común para guardar datos en forma de tabla. Cada fila es un registro y los campos van separados por comas. Para trabajar con CSV en Python usamos el módulo **csv** que viene incluido en la biblioteca estándar. Usamos **csv.DictReader** para leer cada fila como un diccionario, y **csv.DictWriter** para guardar los cambios. También validamos cada fila al cargar el archivo para asegurarnos de que los datos sean correctos antes de usarlos.

2. Decisiones Técnicas y Arquitectura

2.1. Estructura del proyecto

Dividimos el código en tres archivos distintos para que cada uno tenga una responsabilidad clara y sea más fácil de mantener:

Archivo	Qué hace
<code>main.py</code>	Es el punto de entrada. Carga el CSV y muestra el menú principal.
<code>funciones.py</code>	Tiene todas las funciones del sistema: agregar, actualizar, buscar, filtrar, ordenar y estadísticas.
<code>validaciones.py</code>	Funciones para validar lo que ingresa el usuario y las filas del CSV.
<code>países.csv</code>	El archivo con los datos de los países.

2.2. Diagramas de flujo

Para entender mejor cómo funciona el sistema, diseñamos tres diagramas que muestran el flujo del programa desde distintos niveles de detalle.

Flujo general del sistema

Muestra el inicio del programa, la validación del CSV y la navegación principal entre el menú y las operaciones.

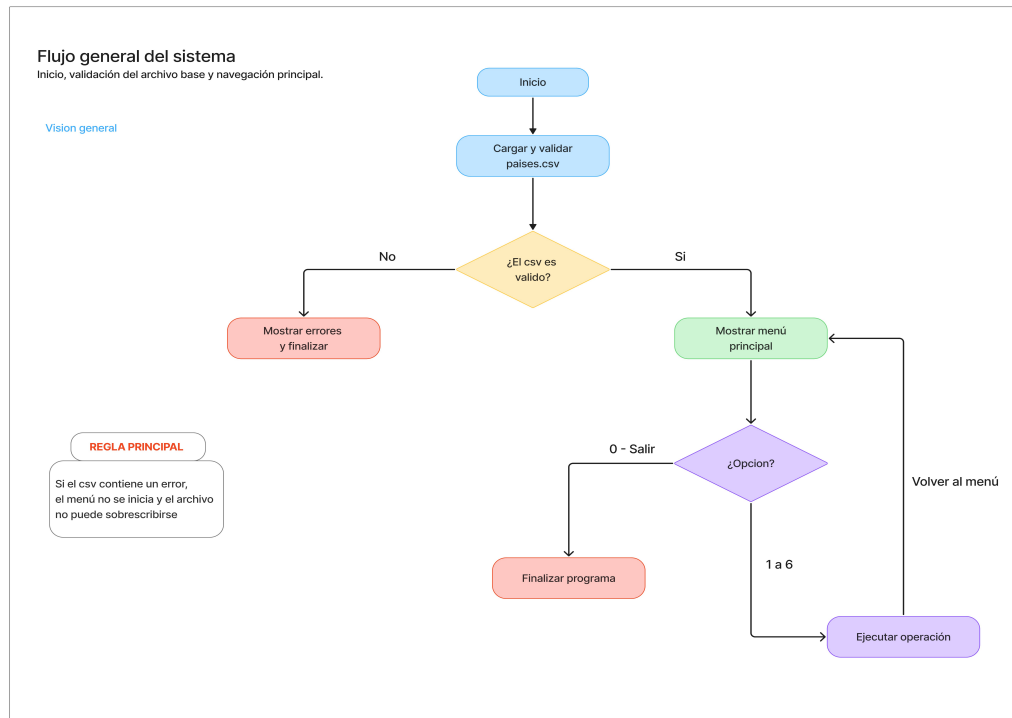


Diagrama 1 — Flujo general del sistema

Carga y validación del CSV

Detalla el proceso interno de la función `cargar_csv()`: cómo abre el archivo, verifica las columnas, recorre cada fila y decide si la carga es válida o no.

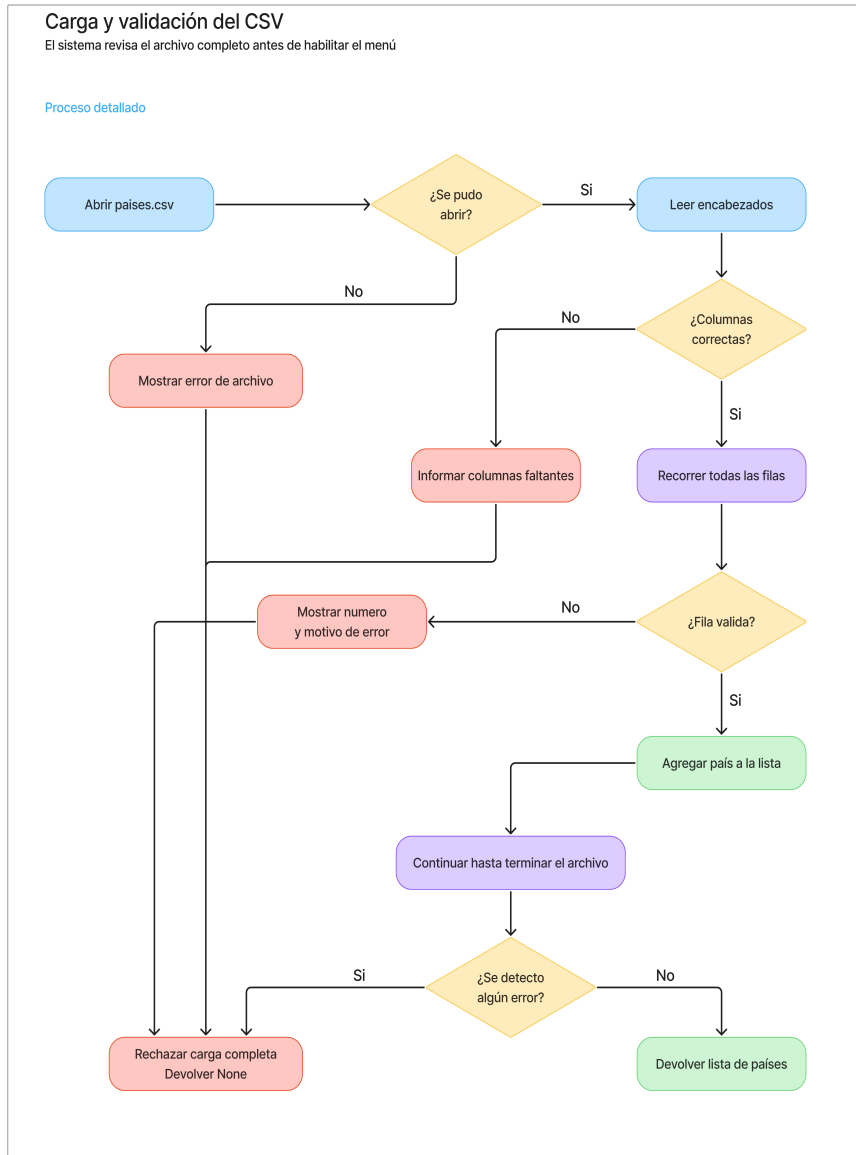


Diagrama 2 — Carga y validación del CSV

Operaciones del menú

Muestra qué hace cada opción del menú, cuáles son de consulta y cuáles modifican el archivo CSV.

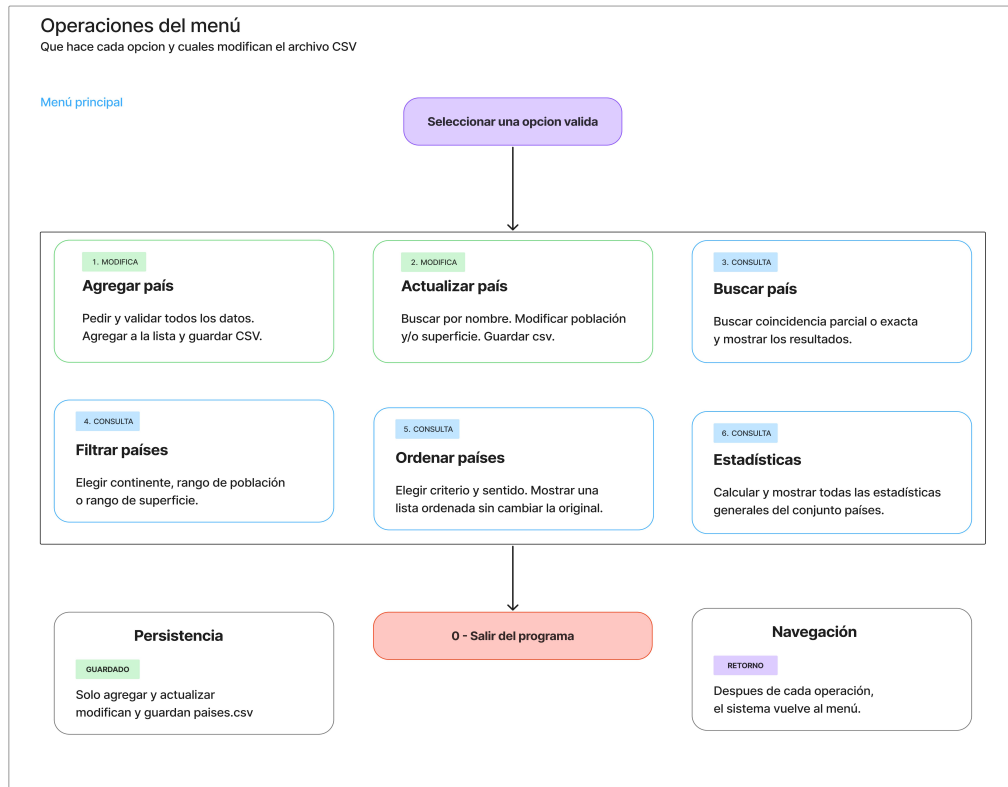


Diagrama 3 — Operaciones del menú

2.3. Ejemplos de ejecución

Ejemplo 1 — Agregar un país:

```
Ingrese el nombre del país: Chile
Ingrese la población del país: 19458310
Ingrese la superficie del país en km²: 756102
Ingrese el continente del país: América
País 'Chile' agregado correctamente.
Los datos se guardaron correctamente.
```

Ejemplo 2 — Buscar por nombre (coincidencia parcial):

```
Ingrese el nombre del país a buscar: arg
-----
Nombre: Argentina
Población: 45376763
Superficie: 2780400 km²
Continente: América
-----
```

Ejemplo 3 — Estadísticas generales:

```
----- ESTADÍSTICAS -----
País con mayor población: China (1412600000 habitantes)
País con menor población: Vaticano (800 habitantes)
Promedio de población: 48350000.00 habitantes
Promedio de superficie: 612000.00 km²

Cantidad de países por continente:
América: 15 | Europa: 20 | Asia: 18 | África: 12 | Oceanía: 5
```

Ejemplo 4 — Validación de entrada incorrecta:

```
Ingrese la población del país: abc
Error: debe ingresar un número entero.
Ingrese la población del país: -500
Error: el número debe ser mayor que cero.
Ingrese la población del país: 1000000
```

3. Dificultades y Conclusiones

Dificultades

Una de las primeras cosas que nos trajo problemas fue la ruta del archivo CSV. Cuando ejecutábamos el programa desde una carpeta distinta a donde estaban los archivos, Python no encontraba el CSV porque lo buscaba en el directorio desde donde se lanzaba el programa, no en el del script. Lo resolvimos usando `os.path.dirname(__file__)` para construir la ruta de forma dinámica, así siempre encuentra el archivo sin importar desde dónde se ejecute.

También nos costó un poco diseñar las validaciones del CSV. Tuvimos que pensar en todos los casos posibles: columnas que faltan, campos vacíos, valores que no son números, problemas de codificación. Al principio los errores aparecían en lugares inesperados y había que rastrear de dónde venían.

Otro punto fue organizar bien los imports entre archivos para no terminar con importaciones circulares. Al final quedó claro: **validaciones.py** no importa nada, **funciones.py** importa de validaciones, y **main.py** importa de los dos.

Conclusiones

Este trabajo nos ayudó a entender mejor cómo funciona un programa real, no solo ejercicios sueltos. Tener que pensar en la estructura completa desde el principio, separar responsabilidades y manejar errores fue bastante diferente a los trabajos anteriores.

Lo que más nos quedó claro es que las validaciones no son un extra: si el programa no controla lo que recibe, falla de maneras difíciles de predecir. Invertir tiempo en eso desde el inicio nos ahorró muchos problemas después.

Trabajar en equipo también tuvo su aprendizaje. Dividir el trabajo por módulos fue útil porque cada uno podía avanzar en su parte sin pisar lo del otro, y al integrar todo fue más ordenado.

4. Bibliografía / Webgrafía

1. Python Software Foundation. (2024). *The Python Tutorial*. <https://docs.python.org/3/tutorial/>
2. Python Software Foundation. (2024). *csv — CSV File Reading and Writing*. <https://docs.python.org/3/library/csv.html>
3. Python Software Foundation. (2024). *Built-in Functions*. <https://docs.python.org/3/library/functions.html>
4. Python Software Foundation. (2024). *os.path — Common pathname manipulations*. <https://docs.python.org/3/library/os.path.html>
5. Real Python. (2024). *Python Dictionaries*. <https://realpython.com/python-dicts/>
6. Real Python. (2024). *Reading and Writing CSV Files in Python*. <https://realpython.com/python-csv/>
7. Lutz, M. (2013). *Learning Python* (5th ed.). O'Reilly Media.

5. Links

Repositorio en GitHub:

https://github.com/kavs2/TPI_programacion-1

Video demostrativo:

[Completar con el link del video una vez grabado]